

Form Data and File Uploads

Send form fields and a file to `upload.php`

Learning Goals

- Distinguish JSON, URL-encoded, and multipart request bodies
- Read ordinary form fields from `$_POST`
- Read uploaded file information from `$_FILES`
- Follow `upload.php` in PHP's execution order
- Recognize upload errors and untrusted file metadata

IMPORTANT

Body Formats

Content type	Typical use	PHP access for POST
<code>application/json</code>	JSON API data	<code>php://input</code> + <code>json_decode()</code>
<code>application/x-www-form-urlencoded</code>	Text-only form fields	<code>\$_POST</code>
<code>multipart/form-data</code>	Form fields and files	<code>\$_POST</code> and <code>\$_FILES</code>

The `Content-Type` header tells PHP how the request body is encoded.

| URL-Encoded POST

Recall from the previous lesson (POST and PUT JSON Requests): `methods.php` accepts more than JSON. It also accepts URL-encoded form data:

```
curl -X POST http://localhost:8000/methods.php \  
  -H "Content-Type: application/x-www-form-urlencoded" \  
  -d "name=Alice&email=alice@example.com"
```

- `-X POST` selects the POST method.
- `-H` identifies the body as URL-encoded form data.
- `-d` sends the `name` and `email` fields.
- PHP decodes these fields and places them in `$_POST`.

URL-Encoded PUT

The same recap applies to PUT:

```
curl -X PUT http://localhost:8000/methods.php \  
  -H "Content-Type: application/x-www-form-urlencoded" \  
  -d "name=Alice+Updated"
```

- `-X PUT` selects PUT instead of POST.
- The header still identifies URL-encoded data.
- `+` represents a space, so the value becomes `Alice Updated`.
- PHP does not populate `$_POST` for PUT; `methods.php` reads

Upload with curl

Start the server in this lesson's code directory:

```
cd code/3_Request_and_Response_with_PHP  
php -S localhost:8000
```

Then send a file and a text field:

```
curl -X POST http://localhost:8000/upload.php \  
-F "document=@README.md" \  
-F "description=Lesson notes"
```

Reading the curl Command

- `-F "document=@README.md"` uploads the local file.
- `-F "description=Lesson notes"` sends a regular form field named `description` with the value `Lesson notes`.
- curl builds the multipart request and boundary automatically.

PHP interprets this curl request like this:

```
$_POST['description'];           // "Lesson notes"  
$_FILES['document']['name'];     // "README.md"  
$_FILES['document']['tmp_name']; // temporary path on the server
```

HTML Form

We can also use an HTML form to upload files.

- HTML forms directly support GET and POST, not PUT.
- File uploads require `multipart/form-data`.
- Form field names become keys in `$_POST` and `$_FILES`.

| Upload with HTML Form

This is a conceptual example.

The HTML form is not in this lesson's code directory; we will build it in the next chapter.

```
<form action="upload.php" method="POST" enctype="multipart/form-data">  
  <input type="text" name="description">  
  <input type="file" name="document">  
  <button type="submit">Upload</button>  
</form>
```

- `action="upload.php"` and `method="POST"` send the form to `upload.php`.
- `enctype="multipart/form-data"` tells PHP that the request contains a file.
- `name="description"` makes the text value available as `$_POST['description']`.
- `name="document"` makes the uploaded file available as the `$_FILES['document']` array.

For example, PHP interprets the submitted form like this:

```
$description = $_POST['description']; // "Lesson notes"
$file = $_FILES['document'];           // an array of file information

// For example, when README.md is selected:
$file = [
    'name' => 'README.md',
    'type' => 'text/markdown',
    'tmp_name' => '/tmp/php1234',
    'error' => UPLOAD_ERR_OK,
    'size' => 1234,
];
```

HTML Form and curl: Same PHP Result

- Both send a `POST` request with `multipart/form-data`.
- HTML uses `<input type="file" name="document">`; curl uses `-F "document=@README.md"`.
- HTML lets the user select a file; `@README.md` tells curl to read a local file.
- In both cases, PHP creates the same `$_FILES['document']` array with `name`, `type`, `tmp_name`, `error`, and `size`.

Reading File Contents

- `$_FILES['document']` contains file information; `tmp_name` is the path to the uploaded bytes.
- PHP reads the contents with `$content = file_get_contents($_FILES['document']['tmp_name']);`.
- `upload.php` returns those contents in the JSON response as `content`.
- The temporary file may be deleted after the request, so move files that must be kept.

IMPORTANT

`upload.php` Execution Flow

1. Require the POST method.
2. Confirm that `$_FILES['document']` exists.
3. Check that the PHP upload status is `UPLOAD_ERR_OK`.
4. Read the temporary file contents.
5. Return the description, content, and file metadata as JSON.

Each failed check stops execution before the success response.

Reading code: `upload.php`

The following slides read `upload.php` top to bottom, in the order PHP executes it.

Step 1: Strict Types and Response Header

```
declare(strict_types=1);  
  
header('Content-Type: application/json');
```

- `declare(strict_types=1)` enables strict type checking if typed code is added.
- The blank line separates setup statements for readability.
- `header(...)` tells the client that every response, including errors, is JSON.

Step 2: Check the Request Method

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    header('Allow: POST');  
    echo json_encode(['error' => 'Use POST']);  
    exit;  
}
```

- The code accepts only POST requests.
- Other methods receive **405 Method Not Allowed** and **Allow: POST**.
- The JSON error is sent, then **exit** stops execution.

Step 3: Read the File Entry

```
$file = $_FILES['document'] ?? null;
if (!is_array($file)) {
    http_response_code(400);
    echo json_encode(['error' => 'Upload a file in the document field']);
    exit;
}
```

- `$file` reads the `document` entry from `$_FILES`; `?? null` handles a missing field.
- `!is_array($file)` detects a missing or invalid upload.
- The server returns `400 Bad Request` as JSON and stops.

Step 4: Check PHP's Upload Error

```
if ($file['error'] !== UPLOAD_ERR_OK) {  
    http_response_code(400);  
    echo json_encode([  
        'error' => 'File upload failed',  
        'upload_error' => $file['error'],  
    ]);  
    exit;  
}
```

- `$file['error']` is `UPLOAD_ERR_OK` when the upload succeeds.
- Any other value returns `400 Bad Request` with the numeric error code as JSON.

Step 5: Read the File Contents

```
$content = file_get_contents($file['tmp_name']);  
if ($content === false) {  
    http_response_code(500);  
    echo json_encode(['error' => 'Could not read the uploaded file']);  
    exit;  
}
```

- `tmp_name` points to the temporary file on the server.
- `file_get_contents()` reads its raw contents into `$content`.
- If reading fails (`$content === false`), the server returns `500 Internal Server Error` and stops.

Step 6: Build the Success Response

```
// Note: Returning raw file content in JSON works for UTF-8 text (e.g., README.md).  
// Binary files (images, PDFs) can fail in json_encode() due to invalid UTF-8 bytes  
// and should be saved directly using move_uploaded_file() or base64-encoded.  
echo json_encode([  
    'description' => $_POST['description'] ?? '',  
    'content' => $content,  
    'file' => [  
        'name' => $file['name'],  
        'size' => $file['size'],  
        'type' => $file['type'],  
        'error' => $file['error'],  
    ],  
], JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
```

- `description`: reads the form field from `$_POST`, defaulting to `''` if omitted.
- `content`: returns the file contents read in Step 5 (assumes UTF-8 text).
- `file`: includes metadata (`name`, `size`, `type`) and the upload status (`error`).
- `JSON_PRETTY_PRINT` formats the output; `JSON_UNESCAPED_SLASHES` preserves `/`.

Example Success Response

```
{  
  "description": "Lesson notes",  
  "content": "# Example notes\n",  
  "file": {  
    "name": "README.md",  
    "size": 1234,  
    "type": "application/octet-stream",  
    "error": 0  
  }  
}
```

The exact **size** and **type** can differ for the file and client.

IMPORTANT

Do Not Trust Client File Metadata

- `$file['name']` can contain an unsafe or misleading filename.
- `$file['type']` is reported by the client and can be false.
- Validate size, content, extension, and destination before saving.
- Binary files should not be directly embedded in JSON without base64 or validation.
- Generate a safe server-side filename instead of using

Test the Error Responses

GET is not allowed and receives **405**:

```
curl -i http://localhost:8000/upload.php
```

A POST without **document** receives **400**:

```
curl -i -X POST http://localhost:8000/upload.php \  
-F "description=No file"
```

-i includes the response status and headers.

IMPORTANT

Upload Response Summary

Situation	Status	Result
Method is not POST	405	JSON error and Allow: POST
document is missing	400	JSON missing-file error
PHP reports an upload error	400	JSON error with upload code
Temporary file cannot be read	500	JSON file-read error
		Description, content, and

Key Takeaways

1. The content type determines how PHP reads a request body.
2. POST form fields appear in `$_POST`.
3. PUT form fields must be parsed from `php://input`.
4. Multipart files appear in `$_FILES` under their field names.
5. Check the method, file entry, and upload error before using metadata.
6. Client filenames and MIME types must not be trusted.